TRƯỜNG ĐẠI HỌC
CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
HCM·UTE
HCMC University of Technology and Engineering

# Chương 5
# TẬP LỆNH ARM
# (Phần 2)



# CONTENT

- Logic Operation Instruction
- Shift and Rotate Instruction
- Data Movement
- Flow Control
- Interrupt & Exception

# Logic Operation Instructions

**Table 4.20 Logic Operation Instructions**

| Instruction | Operation |
|---|---|
| `AND Rd, Rn ; Rd = Rd & Rn`<br>`AND.W Rd, Rn,#immed; Rd = Rn & #immed`<br>`AND.W Rd, Rn, Rm ; Rd = Rn & Rd` | Bitwise AND |
| `ORR Rd, Rn ; Rd = Rd \| Rn`<br>`ORR.W Rd, Rn,#immed; Rd = Rn \| #immed`<br>`ORR.W Rd, Rn, Rm ; Rd = Rn \| Rd` | Bitwise OR |
| `BIC Rd, Rn ; Rd = Rd & (~Rn)`<br>`BIC.W Rd, Rn,#immed; Rd = Rn &(~#immed)`<br>`BIC.W Rd, Rn, Rm ; Rd = Rn &(~Rd)` | Bit clear |
| `ORN.W Rd, Rn,#immed; Rd = Rn \|(~#immed)`<br>`ORN.W Rd, Rn, Rm ; Rd = Rn \|(~Rd)` | Bitwise OR NOT |
| `EOR Rd, Rn ; Rd = Rd ^ Rn`<br>`EOR.W Rd, Rn,#immed; Rd = Rn \| #immed`<br>`EOR.W Rd, Rn, Rm ; Rd = Rn \| Rd` | Bitwise Exclusive OR |

# Shift and Rotate Instructions

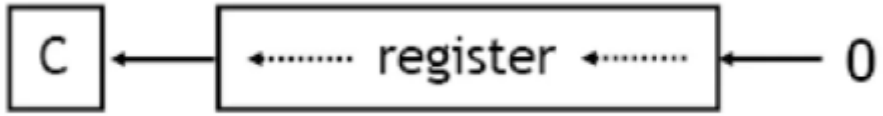
**Table 4.21 Shift and Rotate Instructions**

| Instruction | Operation |
|---|---|
| `ASR Rd, Rn,#immed; Rd = Rn >> immed`<br>`ASR Rd, Rn ; Rd = Rd >> Rn`<br>`ASR.W Rd, Rn, Rm ; Rd = Rn >> Rm` | Arithmetic shift right |

| Instruction | Operation |
|---|---|
| `LSL Rd, Rn,#immed; Rd = Rn << immed`<br>`LSL Rd, Rn ; Rd = Rd << Rn`<br>`LSL.W Rd, Rn, Rm ; Rd = Rn << Rm` | Logical shift left |
| `LSR Rd, Rn,#immed; Rd = Rn >> immed`<br>`LSR Rd, Rn ; Rd = Rd >> Rn`<br>`LSR.W Rd, Rn, Rm ; Rd = Rn >> Rm` | Logical shift right |
| `ROR Rd, Rn ; Rd rot by Rn`<br>`ROR.W Rd, Rn, Rm ; Rd = Rn rot by Rm` | Rotate right |
| `RRX.W Rd, Rn ; {C, Rd} = {Rn, C}` | Rotate right extended |

# Shift and Rotate Instructions

- Dịch trái logic (LSL):

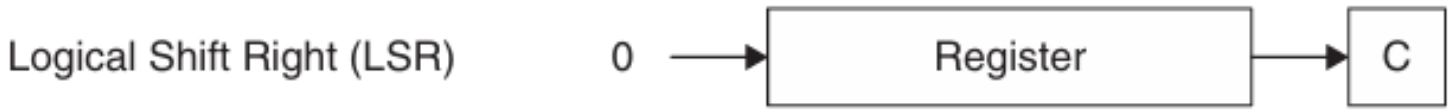



- MOV R0, R2, LSL #2 ;R0 = R2 << 2

- Ví dụ: R2 = 0x43ABF12A
- Đổi sang nhị phân: R2 = 0100 0011 1010 1011 1111 0001 0010 1010
- R0 = 0000 1110 1010 1111 1100 0100 1010 1000

# Shift and Rotate Instructions

- Dịch phải logic (LSR):




- MOV R0, R2, LSR #2 ;R0 = R2 >> 2

- Ví dụ: R2 = 0x43ABF12A
- Đổi sang nhị phân: R2 = **0100 0011 1010 1011 1111 0001 0010 1010**
- R0 = **0001 0000 1110 1010 1111 1100 0100 1010**

# Shift and Rotate Instructions

- Quay phải (ROR)

Rotate Right (ROR)




- MOV R0, R1, ROR #2 ;R0 = R1 rot by #2

- Ví dụ: R1 = 0x00000003
- Đổi sang nhị phân: R1 = 0000 0000 0000 0000 0000 0000 0000 0011
- R0 = 1100 0000 0000 0000 0000 0000 0000 0000

# Shift and Rotate Instructions

- Quay phải mở rộng (RRX)

Rotate Right Extended (RRX)



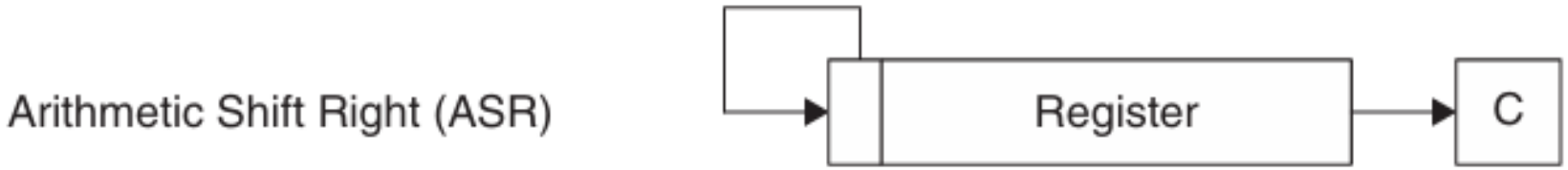

- MOV R0, R2, RRX ; R0 = R2 sau khi quay phải 1 bit
  ; R2 không đổi

- Ví dụ: R2 = 0x00000031; C = 1

- 0000 0000 0000 0000 0000 0000 0011 0001
- Sau: R0 = 0x80000018; C = 1
- R2 = 0x00000031

# Shift and Rotate Instructions

- Dịch phải số học (ASR)




- MOV R0, R2, ASR #1
- Ví dụ: R2 = 0xFFFFFFF0; -16 decimal

- 1111 1111 1111 1111 1111 1111 1111 0000
- 1111 1111 1111 1111 1111 1111 1111 1000
- R0 = 0xFFFFFFF8 ; -8 decimal

# Assembler Language: Processing Data

- For conversion of signed data from byte or half word to word, the Cortex-M3 provides the two instructions shown in Table 4.22.
- Both 16-bit and 32-bit versions are available.
  - The 16-bit version can only access low registers.

**Table 4.22 Sign Extend Instructions**

| Instruction | Operation |
|---|---|
| `SXTB.W Rd, Rm ; Rd = signext(Rn[7:0])` | Sign extend byte data into word |
| `SXTH.W Rd, Rm ; Rd = signext(Rn[15:0])` | Sign extend half word data into word |

# Assembler Language: Processing Data

- Another group of data processing instructions is used for reversing data bytes in a register (see Table 4.23).
- These instructions are usually used for conversion between little endian and big endian data.
  - See Figure 4.2.
- Both 16-bit and 32-bit versions are available.
  - The 16-bit version can only access low registers

**Table 4.23 Data Reverse Ordering Instructions**

| Instruction | Operation |
|---|---|
| `REV.W Rd, Rn ; Rd = rev(Rn)` | Reverse bytes in word |
| `REV16.W <Rd>, <Rn> ; Rd = rev16(Rn)` | Reverse bytes in each half word |
| `REVSH.W <Rd>, <Rn> ; Rd = revsh(Rn)` | Reverse bytes in bottom half word and sign extend the result |

# Assembler Language: Processing Data

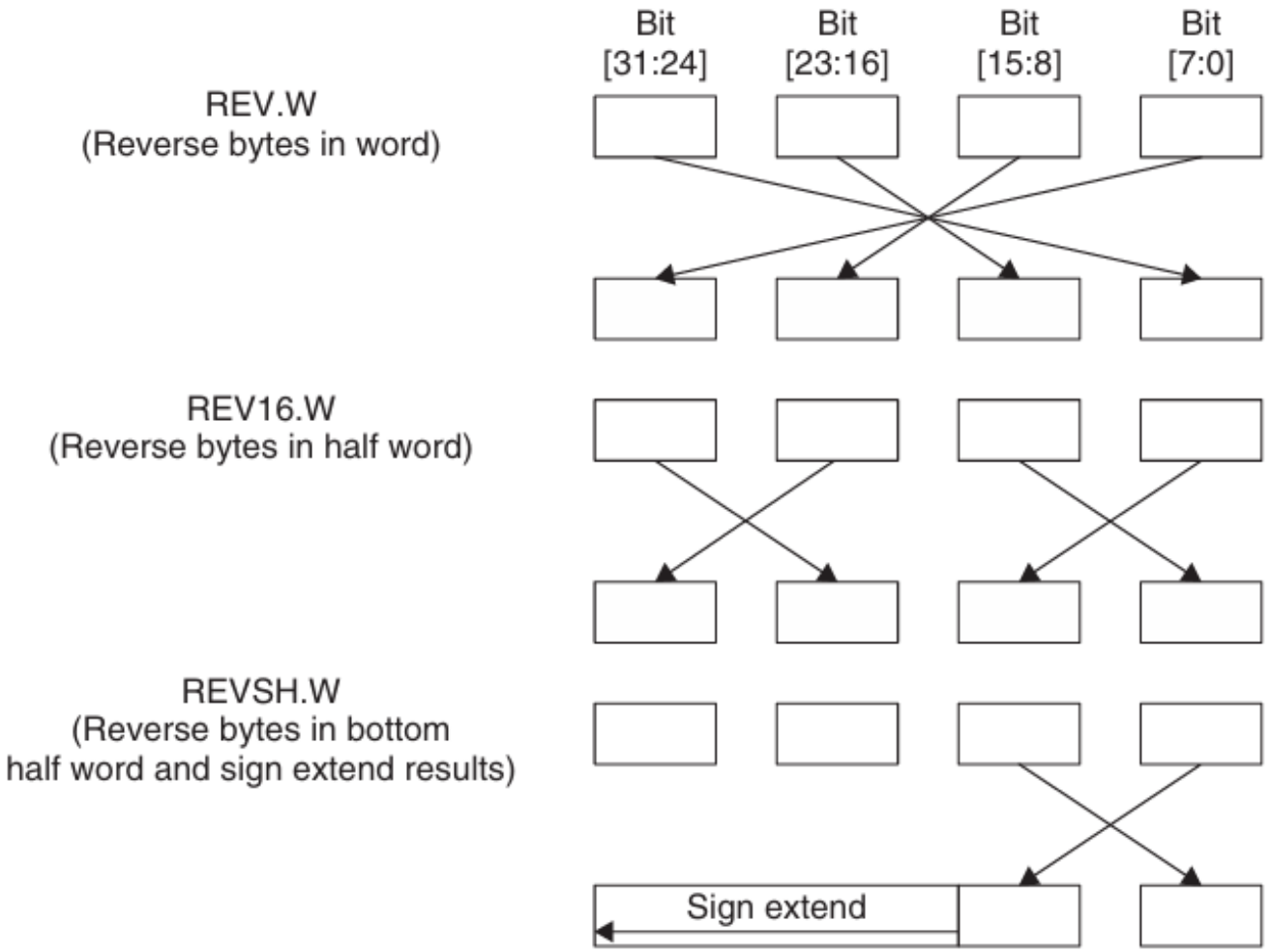



**Figure 4.2 Reverse Operations**

# Assembler Language: Processing Data

- The last group of data processing instructions is for bit field processing.
  - They include the instructions shown in Table 4.24.

**Table 4.24 Bit Field Processing and Manipulation Instructions**

| Instruction | Operation |
|---|---|
| `BFC.W Rd, Rn, #<width>` | Clear bit field within a register |
| `BFI.W Rd, Rn, #<lsb>, #<width>` | Insert bit field to a register |
| `CLZ.W Rd, Rn` | Count leading zero |
| `RBIT.W Rd, Rn` | Reverse bit order in register |
| `SBFX.W Rd, Rn, #<lsb>, #<width>` | Copy bit field from source and sign extend it |
| `UBFX.W Rd, Rn, #<lsb>, #<width>` | Copy bit field from source register |

# Assembler Language: Decisions and Conditional Branches

- With combinations of the four flags (N, Z, C, and V ), 15 branch conditions are defined, as shown in Table 4.26 (page 68 – 69).
- Using these conditions, branch instructions can be written as, for example,

```
BEQ label ; Branch to address ‘label’ if Z flag is set
```

- We can also use the Thumb-2 version if your branch target is further away.
  - For example,

```
BEQ.W label ; Branch to address ‘label’ if Z flag is set
```

## Table 4.26 Conditions for Branches or Other Conditional Operations

| Symbol | Condition | Flag |
|---|---|---|
| EQ | Equal | Z set |
| NE | Not equal | Z clear |
| CS/HS | Carry set/unsigned higher or same | C set |
| CC/LO | Carry clear/unsigned lower | C clear |
| MI | Minus/negative | N set |
| PL | Plus/positive or zero | N clear |
| VS | Overflow | V set |
| VC | No overflow | V clear |
| HI | Unsigned higher | C set and Z clear |
| LS | Unsigned lower or same | C clear or Z set |
| GE | Signed greater than or equal | N set or V set, or<br>N clear and V clear (N == V) |
| LT | Signed less than | N set and V clear, or<br>N clear and V set (N != V) |
| GT | Signed greater than | Z clear, and either N set and V set, or<br>N clear and V clear (Z == 0, N == V) |
| LE | Signed less than or equal | Z set, or N set and V clear, or<br>N clear and V set (Z == 1 or N != V) |
| AL | Always (unconditional) | — |

# Assembler Language: Decisions and Conditional Branches

- Most of the 16-bit Thumb arithmetic instructions affect the N, Z, C, and V flags. With 32-bit Thumb-2 instructions, the ALU operation can either change flags or not change flags. For example:

```
ADDS.W   R0, R1, R2   ; This 32-bit Thumb-2 instruction update flag
ADD.W    R0, R1, R2   ; This 32-bit Thumb-2 instruction do not
                      ; update flag
ADDS     R0, R1       ; This 16-bit Thumb instruction update flag
ADD      R0, #0x1     ; This 16-bit Thumb instruction update flag
```

# Assembler Language: Decisions and Conditional Branches

- The CMP (Compare) instruction subtracts two values and updates the flags (just like SUBS), but the result is not stored in any registers.

```
CMP R0, R1     ; Calculate R0 - R1 and update flag
CMP R0, #0x12  ; Calculate R0 - 0x12 and update flag
```

- A similar instruction is the CMN (Compare Negative). It compares one value to the negative (two's complement) of a second value; the flags are updated, but the result is not stored in any registers:

```
CMN R0, R1     ; Calculate R0 - (-R1) and update flag
CMN R0, #0x12  ; Calculate R0 - (-0x12) and update flag
```

# Assembler Language: Decisions and Conditional Branches

- The TST (Test) instruction is more like the AND instruction. It ANDs two values and updates the fl ags. However, the result is not stored in any register. Similarly to CMP, it has two input formats:

```
TST R0, R1     ; Calculate R0 and R1 and update flag
TST R0, #0x12  ; Calculate R0 and 0x12 and update flag
```

# Assembler Language: Decisions and Conditional Branches

- The IF-THEN (IT) instructions allow up to four succeeding instructions (called an IT block) to be conditionally executed.

```
IT<x>          <cond>
IT<x><y>       <cond>
IT<x><y><z>    <cond>
```

- where:
  - specifies the execution condition for the second instruction
  - specifies the execution condition for the third instruction
  - specifies the execution condition for the fourth instruction
  - specifies the base condition of the instruction block; the first instruction following IT executes if is true
- Each of <x>, <y>, <z> and can be either T (THEN) or E (ELSE), which refers to the base condition <cond> whereas <cond> uses traditional syntax such as EQ, NE, GT, or the like

# Assembler Language: Combined Compare and Conditional Branch

```
if (R0 equal R1) then {
  R3 = R4 + R5
  R3 = R3 / 2
  } else {
  R3 = R6 + R7
  R3 = R3 / 2
  }
```



```
CMP   R0, R1       ; Compare R0 and R1
ITTEE EQ           ; If R0 equal R1, Then-Then-Else-Else
ADDEQ R3, R4, R5 ; Add if equal
ASREQ R3, R3, #1 ; Arithmetic shift right if equal
ADDNE R3, R6, R7 ; Add if not equal
ASRNE R3, R3, #1 ; Arithmetic shift right if not equal
```

# Exercise

1. Cho R0 = 8, R1 = 8. Viết chương trình kiểm tra nếu R0 == R1, thì R2 = 1, ngược lại R2 = 0
2. Cho R0 = 8, R1 = 3.
   - Viết chương trình tìm R2 = max(R0, R1).
   - Nếu R0 > R1 thì R3 = R0 + R1, ngược lại R3 = R0 - R1.
3. Viết chương trình kiểm tra
   - Nếu R0 > 0 thì R1 = 1.
   - Nếu R0 = 0 thì R1 = 0.
   - Nếu R0 < 0 thì R1 = -1.
4. Viết chương trình kiểm tra số chẵn/lẻ. Nếu R0 chẵn thì R1 = 0, ngược lại R1 = 1.

# Exercise

- Cho R0 = A, R1 = B. Viết chương trình:
- Nếu A > B:
  - R2 = A + B
  - R3 = R2 × 4
- Ngược lại:
  - R2 = A - B
  - R3 = R2 / 2

# Assembler Language: Decisions and Conditional Branches

- The compare and then branch if zero/nonzero instructions are useful for looping (for example, the WHILE loop in C). The syntax is:

```
CBZ   <Rn>, <label>
```

or:

```
CBNZ  <Rn>, <label>
```

```
while (R0 != 0) {
    function1();
    }
```



```
    ...
loop
    CBZ R0, loopexit
    BL  function1
    B   loop
  loopexit
  ...
```

# Assembler Language: Combined Compare and Conditional Branch

```
i = 5;
while (i != 0 ){
func1(); ; call a function
i--;
}
```



```
      MOV   R0, #5          ; Set loop counter
loop1 CBZ   R0, loop1exit   ; if loop counter = 0 then exit the loop
      BL    func1           ; call a function
      SUB   R0, #1          ; loop counter decrement
      B     loop1           ; next loop
loop1exit
```

# A TYPICAL DEVELOPMENT FLOW

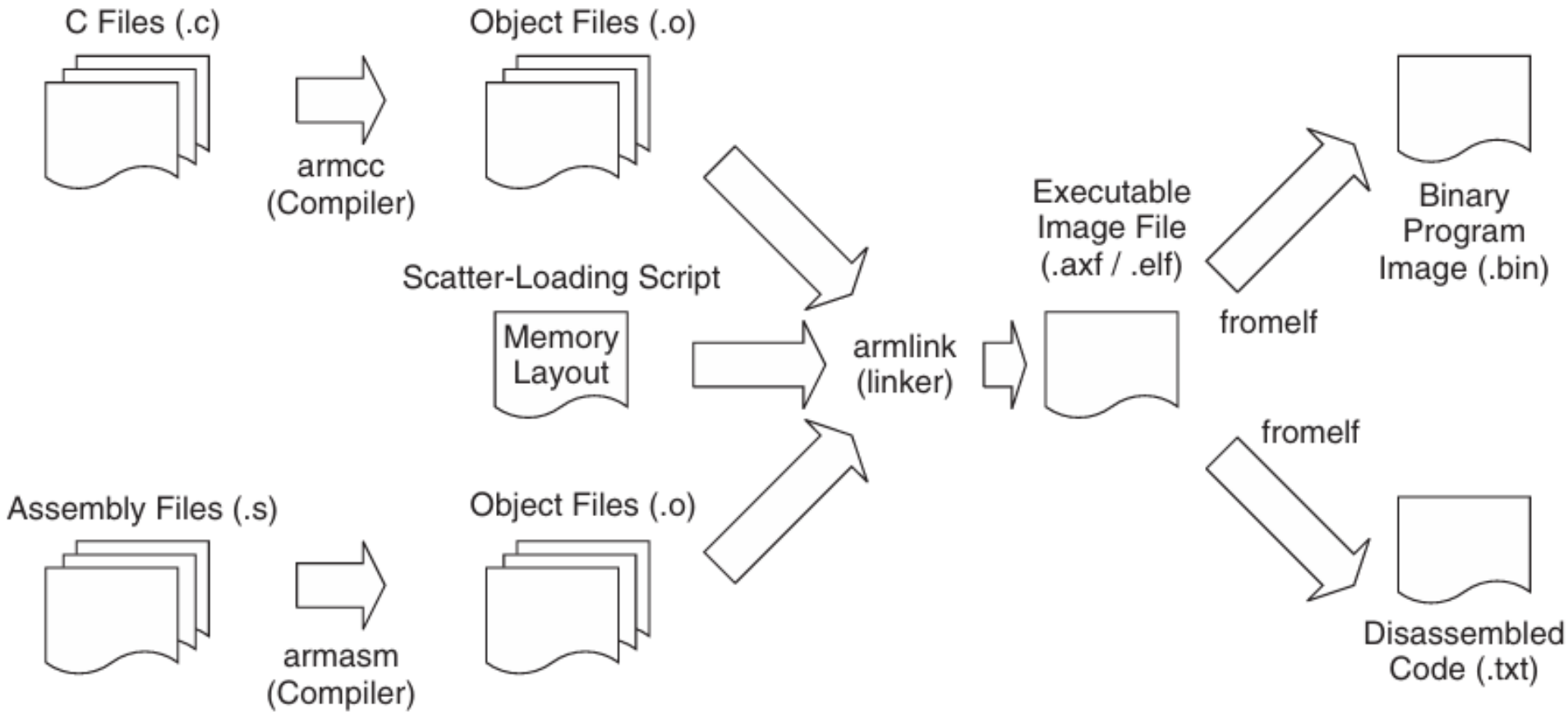



**Figure 10.1 Example Flow Using ARM Development Tools**

# BASIC ARM PROGRAM

```
 AREA    RESET, DATA, READONLY
        EXPORT  __Vectors
        EXPORT  Reset_Handler
__Vectors
        DCD     0x20001000
        DCD     Reset_Handler
        AREA    MyProgram, CODE, READONLY
        THUMB
        ENTRY
Reset_Handler
        <INSTRUCTIONS>
stop
        B       stop
        END
```

TRƯỜNG ĐẠI HỌC
CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
HCM·UTE
HCMC University of Technology and Engineering

# THANK YOU

